Classes and Objects

A class in Java is a user-defined data type that groups data members (variables) and member functions (methods) into a single unit. It serves as a blueprint for creating objects

```
class Student {
    String name;
    int rollNo;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // Creating an object
        s1.name = "Rahul";
        s1.rollNo = 101;
        s1.display();
    }
}
```

Constructors

A constructor is **automatically called when an object is created**. It is mainly used to **initialize the object's data members**.

Characteristics of Constructors

- Constructor name must be the **same as the class name**.
- It has **no return type**.
- It is called **automatically** when an object is created.
- It is used to initialize instance variables.
- Constructors can be **overloaded**.
- Constructors cannot be **inherited**.
- Constructors cannot be **static**, **final**, or **abstract**.

Types of Constructors

1. Default Constructor

If you do **not** write any constructor, Java automatically provides one.

```
class Student {
    int id;
    String name;
}

public class Main {
    public static void main(String[] args) {
```

```
        Student s = new Student();

        System.out.println(s.id);
        System.out.println(s.name);
    }
}

0
Null
```

No-Argument Constructor: A constructor with **no parameters**.

```
class Student {

    Student() {
        System.out.println("Object Created");
    }
}

public class Main {
    public static void main(String[] args) {

        Student s = new Student();
    }
}

Object Created
```

Parameterized Constructor: A constructor that accepts parameters

```
class Student {

    int id;
    String name;

    Student(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student(101, "Rahul");
        Student s2 = new Student(102, "Priya");
```

```
        s1.display();
        s2.display();
    }
}

101 Rahul
102 Priya
```

Copy Constructor

A copy constructor in Java is a constructor that creates a new object by copying the values of another object.

```
class Student {
    int mark;

    // Normal constructor
    Student(int m) {
        mark = m;
    }

    // Copy constructor
    Student(Student s) {
        mark = s.mark;
    }

    public static void main(String[] args) {
        Student s1 = new Student(90);
        Student s2 = new Student(s1);

        System.out.println(s1.mark);
        System.out.println(s2.mark);
    }
}
90
90
```

Constructor Overloading

A class can have **multiple constructors** with different parameter lists

```
class Student {

    Student() {
        System.out.println("No argument Constructor");
    }

    Student(int id) {
        System.out.println("ID = " + id);
    }

    Student(int id, String name) {
```

```
        System.out.println(id + " " + name);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student(101);
        Student s3 = new Student(101, "Rahul");
    }
}

No argument Constructor
ID = 101
101 Rahul
```

Constructor	Method
Same name as class	Any valid name
No return type	Must have a return type (or void)
Called automatically	Called explicitly
Initializes objects	Performs operations
Executes once per object creation	Can be called many times

Constructor Chaining

Constructor chaining means one constructor calls another using this().

```
class Demo {

    Demo() {
        this(10);
        System.out.println("No-Argument Constructor");
    }

    Demo(int x) {
        this(10, 20);
        System.out.println("One-Argument Constructor");
    }

    Demo(int x, int y) {
        System.out.println("Two-Argument Constructor");
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        Demo d = new Demo();
    }
}

Two-Argument Constructor
One-Argument Constructor
No-Argument Constructor
```

this Keyword in Java

this is a reference to the current object of the class

Why is this needed?

```
class Main {

    int d;
    String n;

    Main(int id, String name) {
        d = id;
        n = name;
    }

    void display() {
        System.out.println(d + " " + n);
    }

    public static void main(String[] args) {
        Main s = new Main(101, "Rahul");
        s.display();
    }
}

101 Rahul
```

```
class Main {

    int id;
    String name;

    Main(int id, String name) {
        id = id;
        name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
        Main s = new Main(101, "Rahul");
        s.display();
    }
}

0 null
```

Usage of java this keyword

Refer to the Current Class Instance Variable

```
class Student {

    int id;
    String name;

    Student(int id, String name) {
```

```
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
        Student s = new Student(101, "Rahul");
        s.display();
    }
}

101 Rahul
```

Here,

- this.id → instance variable
- id → constructor parameter

Invoke Another Constructor (this())

```
class Student {

    Student() {
        this(101);
        System.out.println("Default Constructor");
    }

    Student(int id) {
        System.out.println("ID = " + id);
    }

    public static void main(String[] args) {
        new Student();
    }
}

ID = 101
Default Constructor
```

Call the Current Class Method

```
class Demo {

    void display() {
        System.out.println("Display Method");
    }

    void show() {
        this.display();
    }
```

```
    public static void main(String[] args) {
        Demo d = new Demo();
        d.show();
    }
}
```

Display Method

Pass the Current Object as an Argument

```
class Demo {

    void display(Demo d) {
        System.out.println("Current object received");
    }

    void show() {
        display(this);
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        d.show();
    }
}
```

Current object received

Return the Current Object

```
class Student {

    Student getObject() {
        return this;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = s1.getObject();

        System.out.println(s1 == s2);
    }
}
```

true

```
class Student {

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = s1;

        System.out.println(s1 == s2);
    }
}
```

true

```
class Student {
```

```
    Student getObject() {
        return new Student();   // New object
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = s1.getObject();

        System.out.println(s1 == s2);
    }
}

False
Student s1 = new Student();
Student s2 = new Student();
```

this vs Local Variable

```
class Test {

    int x = 10;

    void display() {
        int x = 20;

        System.out.println(x);
        System.out.println(this.x);
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.display();
    }
}

20
10
```

- x → local variable
- this.x → instance variable

Use of this	Example
Refer to current instance variable	this.id = id;
Call current class method	this.display();
Invoke another constructor	this(100);
Pass current object	method(this);
Return current object	return this;

Instance Variable Hiding

```
class Parent {
    int x = 10;
```

```
}

class Child extends Parent {
    int x = 20;    // Hides Parent's x

    void display() {
        System.out.println("Child x = " + x);
        System.out.println("Child x using this = " + this.x);
        System.out.println("Parent x using super = " + super.x);
    }

    public static void main(String[] args) {
        Child c = new Child();
        c.display();
    }
}

Child x = 20
Child x using this = 20
Parent x using super = 10
```

Garbage Collection

Garbage Collection (GC) is the automatic process of **identifying and removing objects that are no longer reachable (used)** by a program, thereby freeing memory in the heap.

Why is Garbage Collection Needed?

Without garbage collection:

- Unused objects would remain in memory.
- Heap memory would gradually fill up.
- The program could eventually throw an OutOfMemoryError.

Garbage Collection automatically:

- Frees unused memory.
- Prevents memory leaks caused by forgotten deallocation.
- Improves application stability

When is an Object Eligible for Garbage Collection?

```
// Nullifying the Reference
public class Main {

    public static void main(String[] args) {

        String s = new String("Java");

        s = null;    // Object becomes eligible for GC
```

```
// Reassigning the Reference
class Student { }

public class Main {

    public static void main(String[] args) {

        Student s = new Student();

        s = new Student();
```

```
        System.gc();

        System.out.println("End of program");
    }
}
```

```
    }
}

//Java has an automatic Garbage Collector.
```

```
// Anonymous Object
class Student {

    Student() {
        System.out.println("Created");
    }
}

public class Main {

    public static void main(String[] args) {

        new Student();
    }
}
```

final

The final keyword is used to restrict modification. It can be applied to variables, methods, and classes.

final Variable

A final variable can be assigned only once

```
public class Main {
    public static void main(String[] args) {
        final int age = 25;
        // age = 30; // Error: Cannot assign a value to final variable
        System.out.println(age);
    }
}

25
```

final Reference Variable

```
final StringBuilder sb = new StringBuilder("Hello");
sb.append(" World"); // Allowed
// sb = new StringBuilder("Hi"); // Error
System.out.println(sb);
```

final method

- If you make any method as final, you cannot override it.
- Is final method inherited? Yes, final method is inherited but you cannot override it.

```
class Bike {

    final void run() {
        System.out.println("running");
    }
}

class Honda extends Bike {

    void run() {
        System.out.println("running safely with
100 kmph");
    }

    public static void main(String[] args) {

        Honda honda = new Honda();
        honda.run();
    }
}

Output: Compile Time Error
```

```
class Bike {

    final void run() {
        System.out.println("Running...");
    }
}

class Honda extends Bike {

    public static void main(String args[]) {

        Honda honda = new Honda();
        honda.run();            // Calls inherited
method
    }
}

Running...
```

final Class

A final class cannot be inherited (extended).

```
final class Vehicle {
    void display() {
        System.out.println("Vehicle");
    }
}

// class Car extends Vehicle { } // Error
```

String Class

```
class StringDemo2 {
    public static void main(String args[]) {

        String strOb1 = "First String";
        String strOb2 = "Second String";
        String strOb3 = strOb1;

        System.out.println("Length of strOb1: " + strOb1.length());
        System.out.println("Char at index 3 in strOb1: " + strOb1.charAt(3));

        if (strOb1.equals(strOb2))
            System.out.println("strOb1 == strOb2");
        else
            System.out.println("strOb1 != strOb2");
```

```
        if (strOb1.equals(strOb3))
            System.out.println("strOb1 == strOb3");
        else
            System.out.println("strOb1 != strOb3");
    }
}

Length of strOb1: 12
Char at index 3 in strOb1: s
strOb1 != strOb2
strOb1 == strOb3
```